JioPC × IIT Bombay

Open Innovation Challenges — Hackathon 2026

Partner: Institute Technical Council, IIT Bombay

Edition: June 2026

About JioPC & The Challenges

Read this first — it applies to all three challenges

Contents

1. About JioPC — Read This First

Read it in full before you start designing.

1.1 What is JioPC?

JioPC is a Desktop-as-a-Service (DaaS) platform built by Jio. It delivers a full Linux desktop to users over the internet — no local PC required. The desktop runs in the cloud and is accessed in real time through the JioPC app on the Jio Set-Top Box (STB).

JioPC is built on a simple but powerful idea: world-class computing should not require world-class hardware. By moving the desktop to the cloud and streaming it to any screen, JioPC brings a full, modern computing experience to anyone with a Jio connection — no high-end device, no expensive software, no compromise.

1.2 VM Configuration — What You Are Building For

Every solution runs on this exact hardware profile. Build and benchmark within these limits — they are non-negotiable.

Component	Specification
CPU	4 vCPU @ 2.45 GHz (shared across the desktop, all running apps, and your solution)
RAM	8 GB
GPU	None — all rendering is CPU-only
Desktop Environment	LxQt on Ubuntu 24.04 LTS
Remote Protocol	RDP — the desktop is streamed to the user's device over a secure gateway

No GPU means every visual effect — animation, shadow, blur — is computed entirely by the CPU. This directly impacts performance for any desktop-layer solution. Design accordingly.

1.3 Floating VM (Virtual Machine) Architecture

JioPC runs on a floating VM model. Every VM in the pool runs an identical OS snapshot called the Gold Image. When a user logs in, a session broker assigns any available VM. The next login may land on a completely different machine — but the user's files and settings follow them. The VM changes; the user's data does not.

```
Login -> broker assigns any idle VM
      -> VM mounts /home/user from NFS
      -> Desktop streamed via RDP
```

Property	What it means for your solution
OS Image	Every VM runs an identical OS image. Your solution ships as a .deb baked into the OS Image — it is available on every VM in the pool.
VM assignment	A user may or may not return to the same VM on their next login. VM-local state cannot be relied upon across sessions.
NFS home (/home/user)	The user's storage is always persistent. Their files, customisation, wallpaper, and apps installed from the Software Center are always available — regardless of which VM they land on. Everything user-specific must live in the user's home directory.
Session reset	All OS processes restart fresh from the Gold Image on every login. User data and preferences are reloaded from the user's home directory.

1.4 No Root Access

Users have no sudo access on the host VM. They cannot install system packages, modify system files, or run privileged services.

- Gold Image integrity — user OS modifications cause VM divergence and break the floating model.
- Infrastructure isolation — VMs share a network with internal Jio systems.
- Multi-tenant safety — one user's changes must never affect the next user on the same VM.

What users can install: Flatpak apps from the curated Software Center. Flatpak apps are always available on the persistent per-user App Disk — when a user installs software, we are simply creating a shortcut so that the app appears in the menu.

1.5 Storage Layout

Rule: anything that must persist across sessions goes into /home/user. Anything system-wide must be packaged as a .deb for the Gold Image build pipeline.

OS Disk (Gold Image)	App Disk (Flatpak)	NFS Home /home/user
Pre-installed apps, system binaries, LxQt session config, XDG autostart entries, and the JioPC desktop shell	Pre-installed Flatpak applications which can be installed from Software Center. Shortcut is created on desktop	User's personal persistent storage, mounted over NFS to that VM when the user logs in;
Read-only to the user — changes are not permitted from end-user.	Per-user, persists across VM changes	Read-write, network-mounted. Roams across all VMs. The only persistent user storage.

2. What You Need to Get Started

2.1 Operating System

All solutions must be developed and tested on Ubuntu 24.04 LTS with the LxQt desktop environment installed. This is the exact OS and desktop configuration running on JioPC.

- Download Ubuntu 24.04 LTS: https://ubuntu.com/download/desktop
- LxQt on Ubuntu: after installing Ubuntu, run "sudo apt install lxqt", or visit https://lxqt-project.org/

2.2 Hardware — Running Ubuntu 24.04 Locally

Your solution must be built and tested inside a virtual machine running Ubuntu 24.04 + LxQt. This is mandatory — solutions must run in a VM environment, and your demo video must show the solution running inside a VM.

Minimum VM spec: 4 vCPU, 8 GB RAM, 50 GB disk.

Use one of the following local virtualisation tools — both are free, support Windows and macOS, and provide a GUI:

Tool	Download	Notes
VMware Workstation Player	https://www.vmware.com/products/workstation-player.html	Free for personal use. Windows and macOS.
VirtualBox	https://www.virtualbox.org/wiki/Downloads	Free and open source. Windows and macOS.

3. How to Test

Your solution must be built and tested inside a virtual machine running Ubuntu 24.04 + LxQt (VMware Workstation Player or VirtualBox). Testing on a bare-metal Linux machine is not sufficient — the VM environment is the target, and your submission must demonstrate this.

Your demo video must show the solution running inside a VM. A recording of a bare-metal install will not be accepted.

Minimum checklist before submitting:

- Fresh Ubuntu 24.04 VM (clean snapshot — no pre-installed dependencies beyond what your INSTALL.md specifies)
- LxQt installed and running as the desktop session inside the VM
- Your .deb installs cleanly with "sudo dpkg -i package.deb"
- All deliverables work as described, including benchmarks, measured inside the VM

4. Submission

A submission form will be shared with all registered participants before the deadline. The form will ask for the following:

4.1 Repository Requirements

- README.md — problem summary, usage guide, benchmark results, known limitations
- design.md — design and architecture, with diagrams depicting the solution
- INSTALL.md — step-by-step installation on a fresh Ubuntu 24.04 + LxQt VM
- src/ — all source code
- packaging/ — .deb package or a build-deb.sh script to build it
- benchmarks/ — methodology and results
- screenshots/ — screenshots of the solution running inside a VM
- video/ — demo video (or a link to it) showing the solution running inside a VM

4.2 Submission Form Fields

Field	Required?	Details
Team name	Required	
Team members	Required	Name, email, and institute roll number for each member
Challenge selected	Required	CHALLENGE-01, 02, or 03
GitHub repository URL	Required	Must be public and contain README.md and INSTALL.md
Solution description	Required	Max 300 words — what you built and what problem it solves
Demo video link	Required	Can be on Google Drive or in the Git repository, max 3 minutes — must show solution running inside a VM

Attachment	Optional	PDF report or architecture diagram, max 10 MB

5. Evaluation Criteria

All challenges are evaluated on the same five dimensions. Evaluation maps directly to the deliverables listed in each challenge document. Each challenge document also contains a specific Yes/No acceptance checklist (the EVALUATE checklist) and challenge-specific success metrics.

Criterion	Weight	Tied to deliverable
Functionality	30%	Does it satisfy all items in the SUBMIT checklist? Does it pass all items in the EVALUATE checklist?
Performance	25%	Do benchmark results meet the stated Success Metrics? Is the methodology documented and reproducible?
Code Quality & Security	20%	Source code: readable, no hardcoded values, easy to extend. Justified technology choices in the design document.
Documentation	15%	README.md completeness, INSTALL.md accuracy, architecture diagram, honest limitations section.
Innovation / Bonus	10%	Bonus goals achieved. Creative approaches beyond stated scope.

Automatic Disqualification

- Solution which requires user to have root access during normal operation
- Solution requires a GPU to function
- .deb does not install on a fresh Ubuntu 24.04 + LxQt system
- GitHub repository is private or missing INSTALL.md
- Benchmark methodology is absent or not reproducible

6. The Three Challenges

Each challenge has its each challenge has its own dedicated document (Documents 2–4) with the full background, constraints, recommended tools, evaluation criteria, and deliverables.

Challenge	Difficulty	Max points	Description
CHALLENGE-01 JioPC Home — Engaging Desktop Experience	Hard	100 pts	Build a lightweight desktop shell featuring a Mac-style dock, application menu, live CMS-driven widgets, a theme engine, and a first-time onboarding wizard.
CHALLENGE-02 Automated Testing	Medium - Hard	75 pts	Build a scripted validation agent that tests web apps, native app health, and start

Agent			menu/desktop app presence on a freshly patched JioPC OS Image, and send a testing report.
CHALLENGE-03 Session Hibernate	Hard	100 pts	Automatically capture all running GUI app state on user disconnect and restore it on the next login — on any VM in the pool — using per-app restore handlers and a notification-driven confirm flow, without requiring OS-level hibernate or root access.

Technical reference and useful links will be provided in separate challenge documents. For questions or clarifications, contact the Jio team before finalising your design.